HealthApp

מסמך תכנון ועיצוב

גרסה 1.0

מגישה: הודיה ויצמן

תוכן עניינים

1. תיאור כללי של המערכת 3
2. הנחות עבודה בכתיבת הפרויקט 3
3. מוסכמות רישום 4
4. הנחות 4
5. שכבת בסיס הנתונים 4
6. שכבת הלוגיקה 5

עקרונות מנחים בשכבת הלוגיקה *5*
FoodItem 6
FoodToday 6
GoalItem 7
MyValue 8

מחלקות שירות:(Services) *9*
MyService 9
FoodService 10
FoodTodayService 12
:GoalService 13
MyProfileService 15

7. שכבת ההצגה(Presentation Layer) 18

עקרונות מנחים בשכבת ההצגה: *18*
Profile 22
Goals 23
EatToday 25
FoodsHistory 27
FoodsList 29

8. דפוסי עיצוב 33

Singleton 33
MVVM (Model-View-ViewModel) 34
Observer 34

9. שינויים עתידיים אפשריים 35
10. דיאגרמות 36

class diagram 36

1. תיאור כללי של המערכת

המערכת בנויה על סמך מודל 3-Tier, שבו המערכת מופרדת לשלוש שכבות:

- שכבת בסיס הנתונים (Database Layer)
  שימוש בJson. שכבה זו אחראית על ההתקשרות עם מאגר המידע, טעינה ושמירה של נתונים. היא מספקת ממשק לשכבות העליונות לביצוע פעולות על גבי בסיס הנתונים.
- שכבת הלוגיקה (Business Logic Layer)
  שכבה זו אחראית על יישום הפונקציונליות של המערכת, מבצעת חישובים, מעבדת ומנתחת את הנתונים משכבת הDatabase.
- שכבת התצוגה (Presentation Layer)
  אחראית על ממשק המשתמש, הצגת הנתונים,, עיצוב, ניווט והאינטראקציה במערכת. מטפלת בקלט מהמשתמש ומעבירה אותו לשכבת הלוגיקה.

כל שכבה מתקשרת עם השכבה מעליה באמצעות מימוש קבוע מראש.

ההפרדה לשכבות מאפשרת לנו ניהול קל יותר של היישום.

מאפשרת לנו להחליף בעתיד את הממשק בין שתי שכבות מבלי להחליף את הממשק בין שתי השכבות האחרות, וכן, להחליף את המימוש של אחת השכבות מבלי להחליף את המימוש של השכבות האחרות, ובכך לחסוך בשכתוב קוד קיים ולאפשר גמישות לשינויים, תחזוקה, הרחבה ובדיקה של המערכת.

2. הנחות עבודה בכתיבת הפרויקט

1. בכל רגע יש מופע יחיד של התוכנה
2. המערכת הינה המלצה בלבד והינה באחריותו הבלעדית של המשתמש
3. לכל מאכל כללי יש ID חיובי בלבד. ID שלילי שמור למאכלים אישיים שיתווספו ע"י המשתמש במידת הצורך.
4. ישנו חיבור יציב לאינטרנט במשך טעינת הנתונים.
5. שמות המאכלים לא מכילים יותר מ 200 תווים
6. ערכים תזונתיים עבור המאכלים בטווח ערכים של 0 עד 100. אנרגיה עד 1000.
7. שלמות הנתונים:

- המערכת מניחה שכל הקבצים הרלוונטיים מאוחסנים בשלמותם במיקום המיועד, ולא יימחקו או ייפגעו במהלך פעולתה.
- במידה וקובץ מסוים נמחק או ניזוק, המערכת אינה יכולה לוודא את תקינות הנתונים הנותרים ועלולה להציג את המאכלים בצורה לא מדויקת,

3. מוסכמות רישום

1. כל שם מחלקה/שיטה/שדה שכוללת יותר ממילה אחת, נכתבת עם אות גדולה בתחילת כל מילה נוספת.
   - מלבד השדות ב FoodItem שבהן נכתב קו תחתון (_) בתחילת כל מילה נוספת.
2. כל מחלקה ושיטה מתחילים באות גדולה.
3. כל שדה סטטי נכתב באותיות גדולות. נכתב קו תחתון (_) בתחילת כל מילה נוספת.
4. פרמטרים לשיטה או משתנים בתוך שיטה מתחילים באות קטנה.
5. Struct, Enum והפרמטרים בתוכם מתחילים באות גדולה.
6. לכל המחלקה המשמשת לשירות נוספה המילה service בסוף שם המחלקה.

4. הנחות

- ישנו חיבור יציב לאינטרנט במשך טעינת הנתונים.
- שמות המאכלים לא מכילים יותר מ 200 תווים
- ערכים תזונתיים עבור המאכלים בטווח ערכים של 0 עד 100. אנרגיה עד 1000.
- המערכת שומרת קבצים על בסיס הנתונים של המשתמש, המערכת אינה תומכת במחיקה של רובץ אחד בלבד.

5. שכבת בסיס הנתונים

שכבת הנתונים אחראית על אחסון, עיבוד, וניהול המידע של האפליקציה.

במקרה זה, שימוש בפורמט Json (JavaScript Object Notation) מעניק יתרונות רבים כמו מבנה קל לקריאה וכתיבה, סטנדרט נתונים מוכר, התאמה לשפות תכנות רבות, פשטות וגמישות גבוהה המעניקים יכולת התרחבות. בנוסף Json הוא בעל משקל קל ותופס פחות נפח משיטות אחסון מוכרות אחרות

החסרונות בשיטה זו הם חוסר תמיכה במבנים מורכבים מאוד, תמיכה רק בסוגי נתונים פשוטים: מחרוזות, מספרים, מערכים, ואובייקטים, אין תמיכה ישירה בסוגי נתונים כמו תאריכים ולכן יש צורך לבצע עיבוד נוסף, עלול להיות פחות יעיל באחסון נתונים גדולים בהשוואה למבני נתונים בינאריים, Json עצמו אינו מציע מנגנוני אבטחה כמו הצפנה או חתימה דיגיטלית, רגיש להתקפות.

במערכת זו ישנו שימוש בקובץ Json סטטי הנמצא על השרת ובקובץ Json דינמי הניתן לשינוי עם השימוש באפליקציה.

שכבת הנתונים תקרא ותעדכן קבצי JSON באופן מקומי.

קובץ ה Json הסטטי נמצא בשרת, מיועד לטעינת המאכלים הכללים. מבחינת תחזוקה וניהול גרסאות - היתרון הוא שניתן לעדכן את המאגר בכל עת, אך החיסרון הוא שקשה לעדכן נתונים בקובץ JSON גדול תוך שמירה על עקביות.

6. שכבת הלוגיקה

שכבה זו מקשרת בין שכבת בסיס הנתונים לשכבת ההצגה, והיא מהווה הפשטה של האובייקטים הקיימים במערכת. כל אובייקט מיוצג באמצעות מחלקה משלו. מחלקות אלו יתוארו בהמשך.

עקרונות מנחים בשכבת הלוגיקה

- בעת פניה לשכבת בסיס הנתונים (Dal), שכבת הלוגיקה תספק את בדיקות אימות הנתונים (Validation) וכן, בדיקות חריגות (Exceptions) עבור הפלטים שמתקבלים ממנה.
- עקרון הכימוס (Encapsulation) בא לידי ביטוי בשכבת הלוגיקה, בכך שאנו מסתירים את המידע הנשמר במחלקות וחושפים אותו כלפי חוץ באמצעות תכונות (Properties) בלבד. מעבר ליתרונות הידועים של עקרון הכימוס, הקוד הופך לגמיש יותר, מהבחינה שבמידה וירצו להוסיף הגבלות מסוימות על רכי המידע או בדיקות ניתן יהיה להוסיף זאת לכל מידע שנשמר וזאת מבלי צורך לשכתב קוד קיים במקומות אחרים
- עקרון ה-(Single Responsibility Principle) SRP יבוא לידי ביטוי בשכבת הלוגיקה בכך שכל מחלקה תקבל אחריות אחת בלבד.
- עקרון ה-(Liskov Substitution Principle) LSP יבוא לידי ביטוי בשכבת הלוגיקה בכך שכאשר המימוש יאפשר זאת, אז פונקציות העושות שימוש במצביעים ל-Base Class יהיו מסוגלות לעשות שימוש ב-Derived Classes מבלי לדעת על כך.
- עקרון ה-Open-Closed Principle יבוא לידי ביטוי בכך שהקוד ייבנה בצורה מודולרית וגמישה תוך תמיכה בהרחבות עתידיות, כך שמודולים יהיו סגורים לשינויים, אך ניתנים להרחבה.
- שימוש בדפוסי עיצוב (Design Patterns) רלוונטיים כדי לוודא שעיצוב של שכבה זו יהיה גמיש לשינויים ויעמוד ביעדי הנדסת תוכנה נוספים.

כעת נעבור על המחלקות השונות.

מודל התחום(Domain Model)

FoodItem

מחלקה זו מייצגת פריט מזון במערכת, כולל מספר מזהה, שם , וערכים תזונתיים.

תכונות

שם המאפיין	סוג	תיאור
Id	Int	מספר מזהה של פריט המזון
Name	String	שם פריט המזון
Protein	double	חלבון (גרם) ב100 גרם
Total_fat	double	שומנים (גרם) ב100 גרם
Carbohydrates	double	פחממות (גרם) ב100 גרם
Energy	Int	קלוריות ב100 גרם
Total_sugars	double	סוכרים (גרם) ב100 גרם
Sodium	double	נתרן (מיליגרם) ב100 גרם

שיטות

שם השיטה	פרמטרים	מחזירה	תיאור
CopyFoodItem(FoodItem food)	food- מאכל מסוג FoodItem	void	מקבלת מאכל ומעתיקה את הפרטים למאכל הקורא לשיטה

FoodToday

מחלקה זו מייצגת פריט מזון שנאכל ביום מסוים, כולל מזהה הפריט, הכמות שנצרכה בתאריך הספציפי, וחישובים המבוססים על הכמות שנצרכה.

תכונות

שם המאפיין	סוג	תיאור
FoodItemId	Int	מספר מזהה של פריט המזון שנאכל
Amount	Int	כמות שנאכלה בגרם
Date	DateTime	תאריך אכילה

בנאים

שם הבנאי	פרמטרים	תיאור
FoodToday(int foodItemId, int amount)	foodItemId: מזהה הפריט amount: כמות שנצרכה	יוצר אובייקט חדש של פריט מזון עם מספר מזהה, כמות, ותאריך ברירת מחדל (היום).

שיטות

שם השיטה	פרמטרים	מחזירה	תיאור
ValueByAmount(double value)	Value: ערך (תזונתי) המבוסס על 100 גרם	double	מחזירה את הערך המותאם על פי הכמות שנצרכה.

GoalItem

מחלקה זו מייצגת מטרה כללית. כולל שם המטרה, מצבה הפעיל מטרות קלוריות וחלבון עבור המטרה הכללית ומבנים המסייעים להגדרת מטרות אלו

מבנים

:MinMax (enum)

מייצג את סוג היעד

Minimum/Maximum

Target(struct)

בעל השדות הבאים:

Minmax מסוג MinMax מציין את סוג המטרה

Num מסוג double מציין את הערך המספר של היעד

תכונות

שם המאפיין	סוג	תיאור
Name	String	שם המטרה
IsActive	Bool	אינדיקציה האם המטרה פעילה
CalorieTarget	Target	יעד קלורי שנקבע
ProTarget	Target	יעד חלבון שנקבע

בנאים

שם הבנאי	פרמטרים	תיאור
GoalItem(string name, Target calorieTarget, Target proTarget)	name – שם המטרה calorieTarget – יעד קלורי proTarget – יעד חלבון	יוצר אובייקט חדש עם שם ויעדים שנקבעו

MyValue

מחלקה זו מייצגת את הפרטים האישים של המשתמש הנמדדים בערך מספרי (כגון משקל). כוללת שם מספר ויחידות

תכונות:

שם המאפיין	סוג	תיאור
Name	string	שם התכונה
Num	int	ערך מספרי
Units	string	יחידות מידה

בנאים

שם הבנאי	פרמטרים	תיאור
MyValue(string name, string units)	name – שם התכונה Units – יחידות מידה	יוצר אובייקט חדש עם שם ויחידות מידה.

מחלקות שירות:(Services)

מחלקות אלו מטפלות בתהליכים עסקיי, שלא שייכים ישירות לאובייקט בודד במודל התחום, הן אחראיות לחשיפת פעולות לשימוש שכבת התצוגה.

תלויות(Dependencies)

- **JsonConvert (Newtonsoft.Json)**
  לטיפול בנתוני JSON עבור קריאה וכתיבה.
- **File System**
  לשמירה וטעינת נתונים של מאכלים אישיים מקובץ מקומי.

MyService

מחלקה אבסטרקטית. מחלקה זו מכילה את הלוגיקה הכללית המשותפת לכל מחלקות השירותים.

תכונות

שם המאפיין	סוג	תיאור
FileS	string	כתובת הקובץ המכיל את הנתונים (path)
_INSTANCE	class	מופע המחלקה היחיד
INSTANCE	class	מופע ציבורי עבור גישה לINSTANCE_
OnChange	Action	אירוע עבור הפצת שינויים

בנאים

שם הבנאי	פרמטרים	תיאור
MyService()	אין	יוצר אובייקט חדש של שירות.

שיטות אבסטרקטיות

שיטות אבסטרקטיות אלו נועדו להבטיח שכל מחלקת שירותים היורשת ממחלקה זו תספק את השיטות הנ"ל בהתאם לצרכיה.

שם השיטה	פרמטרים	מחזירה	תיאור
InitializeList ()	אין	void	מאתחלת רשימה מקומית
LoadItemsToTheList(string itemJson)	itemJson: מחרוזת המכילה את הרשום בקובץ המדובר	void	טוען את הנתונים שבמחרוזת לרשימה הרצויה
SaveItems()	אין	Void	שמירת הנתונים בקובץ FileS

ממשקים ציבוריים(Public Interfaces)

שם השיטה	פרמטרים	מחזירה	תיאור
SetFileName(string fileName)	fileName: שם הקובץ	void	שמירה בFileS את כתובת הקובץ עבור הנתונים
LoadItems ()	אין	void	מחליט האם לטעון לרשימה או לקרוא לרשימה התחלתית
Save()	אין	void	שומר את הנתונים ומוציא הודעת שמירה
MyRound (double num)	num: ערך מספרי	double	עיגול המספר לתצוגה
DaysApart(DateTime day)	day: יום לבדיקה	Double	מחזירה את הפרש הימים
int StringToInt(object? obj)	Obj: האובייקט להמרה	Int	ממירה אובייקט לint
(StringToIntOrNull (object? Obj)	Obj: האובייקט להמרה	Int או null	ממירה אובייקט לint או null
NotifyStateChanged()	איו	Void	מפיצה את השינויים למנויים

מחלקה זו מכילה שיטות ציבוריות נוספות המיועדות לתצוגה כגון ProString וכדומה

FoodService

מחלקה זו אחראית לעיבוד ושמירת נתונים, לניהול אובייקטים הקשורים לנתוני מזון ולחישובים נוספים

המחלקה יורשת מהמחלקה MyService ומרחיבה את הפונקציונליות שלה.

תכונות

שם המאפיין	סוג	תיאור
Foods	List<FoodItem>	רשימת המאכלים לבחירה הקיימים במערכת
FoodsPersonal	List<FoodItem>	רשימת מאכלים אישיים שהתווספו לרשימת המאכלים ע"י המשתמש

בנאים

שם הבנאי	פרמטרים	תיאור
FoodService()	אין	השמה של FileS לכתובת הרצויה ואתחול אובייקט HttpClient להמשך הפעילות במחלקה.

ממשקים ציבוריים(Public Interfaces)

שם השיטה	פרמטרים	מחזירה	תיאור
LoadItems()	אין	משימה אסינכרונית (Task).	בנוסף למתואר במחלקת האב, טוענת רשימת מאכלים ממקור חיצוני ומעדכנת רשימה מקומית
LoadItemsToTheList(string itemJson)	itemJson: מחרוזת JSON לטעינת מאכלים	void	טעינת מאכלים אישיים מitemJson לרשימה מקומית
AddPersonalFood(FoodItem food)	food: מאכל אישי	void	הוספת מאכל אישי לרשימות המקומיות
InitializeList()	אין	void	מאתחלת רשימת מאכלים אישיים ריקה
SaveItems()	אין	Void	שומרת את רשימת המאכלים האישיים בקובץ
GetFreeId()	אין	Int	מחזירה מספר מזהה פנוי לשימוש במאכלים אישיים
SumFoodsValues(List<FoodToday> foodTodayL)	foodTodayL: רשימת אובייקטים מסוג FoodToday	ערכים תזונתיים (double, int)	מחזירה את סכום הערכים התזונתיים עבור רשימת המאכלים
FoodTodayString(FoodToday food)	food: אובייקט מסוג FoodItem	string	מחזירה מחרוזת להצגת המאכל
FoodTitleForAmount(FoodToday food)	food: אובייקט מסוג FoodItem	string	מחזירה כותרת עבור הצגת הערכים של המאכל
FoodDetailsString(FoodToday foodT)	foodT: אובייקט מסוג FoodItem	String	מחזירה פירוט מלא של ערכים תזונתיים עבור המאכל

תלויות נוספות (Dependencies)

- **HttpClient**

משמש לשליחת בקשות HTTP לאחזור נתוני מאכלים ממקור חיצוני.

FoodTodayService

מחלקה זו אחראית על ניהול המים והמאכלים שנאכלו, היא מכילה בתוכה מבנים שיעזרו לניהול זה,

מחלקה זו יורשת מMyService ותרחיב את הפונקציונליות שלה

מבנים

GlassOfWater (struct)

בעל השדות הבאים:

Day מסוג DateTime מציין את היום המדובר

NumOfGlassמסוג int מציין את מספר הכוסות עבור אותו יום

תכונות

שם המאפיין	סוג	תיאור
DAYS_FOR_SAVE	קבוע מסוג int	מציין את מספר הימים לשמירת היסטוריה
FoodsToday	רשימת אובייקטים מסוג FoodToday	רשימת המאכלים שנאכלו
FileWater	string	כתובת הקובץ המכיל את נתוני המים
GlassOfWaterList	רשימת אובייקטים מסוג GlassOfWater	רשימה המכילה את נתוני השתיה
FService	FoodService	מופע מחלקה FoodService

בנאים

שם הבנאי	פרמטרים	תיאור
FoodTodayService()	אין	השמה של FileS ו FileWater לכתובות הרצויות עבור המשך הפעילות במחלקה.

ממשקים ציבוריים(Public Interfaces)

שם השיטה	פרמטרים	מחזירה	תיאור
LoadItems()	אין		בנוסף למתואר במחלקת האב, דואגת לאתחול נתוני שתיה
LoadItemsToTheList(string itemJson)	itemJson: מחרוזת JSON לטעינת מאכלים	Void	טעינת מאכלים שנאכלו מitemJson לרשימה מקומית
InitializeList()	אין	Void	מאתחלת רשימה ריקה של מאכלים שנאכלו
SaveItems()	אין	Void	מוחקת נתונים ישנים ושומרת את רשימת

			המאכלים שנאכלו ואת נתוני השתיה בקבצים
NumGlassBeforNumOfDays(int beforNumOfDays)	beforNumOfDays : מספר ימים	Int	מקבלת מספר ומחזירה את מספר כוסות המים עבור היום המבוקש (לפני מספר ימים שהתקבל)
AddGlassOfWater()	אין	Int	מוסיפה כוס למספר הכוסות היום ומחזירה את המספר החדש
FoodsBeforeNumOfDays (int days)	days: מספר ימים	רשימת אובייקטים מסוג FoodToday	מקבלת מספר ומחזירה את כל המאכלים שנאכלו עבור היום המבוקש(לפני מספר ימים שהתקבל)
AddFood(int foodId, int amount)	foodId: מזהה מאכל amount: הכמות שנאכלה	Void	מאתחל מאכל חדש מסוג FoodToday ומוסיף אותו לרשימה המקומית
TotalValuesBeforNumOfDays(int beforeNumOfDays)	beforeNumOfDays: מספר ימים	ערכים תזונתיים (double, int)	מקבלת מספר ומחזירה את הערכים התזונתיים עבור היום המבוקש (לפני מספר ימים שהתקבל)

GoalService:

מחלקה זו אחראית על ניהול המטרות הכלליות חישוב ולוגיקה עבור המטרות היומיות, היא מכילה בתוכה מבנים שיעזרו לניהול זה,

מחלקה זו יורשת מMyService ותרחיב את הפונקציונליות שלה

Delegate

נעשה שימוש בdelegate שחתימתה:

public delegate Target CalculateMyTarget(int a, GoalItem b)

מטרתה היא להצביע על הפונקציה לחישוב מטרה יומית. פונקציה צריכה לקבל מטרה כללית ונתון מספרי המיועד לחישוב המטרה היומית.

לדוגמה חישוב מטרה יומית עבור כמות חלבון לצריכה, תלויה במשקל ובמטרה הכללית של המשתמש.

פונקציה זו תהיה חלק ממבנה TargetData המתואר בהמשך.

במידה ובהרחבה עתידית נרצה להוסיף מטרות יומיות התלויות בפרמטרים נוספים יהיה נדרש להוסיף פרמטרים או לשנות Delegate זה.

מבנים

Target (struct)

מוגדר בדומה להגדרה בGoalItem. מיועד להגדרת המטרות היומיות. בעל השדות הבאים:

Minmax מסוג MinMax המוגדר במחלקה GoalItem מציין את סוג היעד מינימום או מקסימום

Num מסוג double מציין את היעד המספרי

TargetData (struct)

מיועד לשמור מידע עבור מטרות יומיות. בעל השדות הבאים:

Priorities: מכיל רשימת מחרוזות המגדיר את העדיפות המטרות הכלליות לחישוב מטרה יומית זו.

Calculate: מכיל דילגט מסוג CalculateMyTarget עבור הפונקציה שתחשב מטרה יומית זו.

Unit: מחרוזת עבור יחידות מידה.

תכונות

שם המאפיין	סוג	תיאור
Goals	רשימת אובייקטים מסוג GoalItem	רשימת המטרות הכלליות
PRO_TARGET	Target	מטרת חלבון יומית
CAL_TARGET	Target	מטרת קלוריות יומית
ProTargetData	TargetData	נתונים עבור מטרת חלבון יומית
CalTargretData	TargetData	נתונים עבור מטרת קלוריות יומית

בנאים

שם הבנאי	פרמטרים	תיאור
GoalService()	אין	השמה של FileS לכתובת הרצויה

ממשקים ציבוריים(Public Interfaces)

שם השיטה	פרמטרים	מחזירה	תיאור
LoadItemsToTheList(string itemJson)	itemJson: מחרוזת JSON	Void	אתחול מטרות כלליות כפי שנשמרו בקובץ
InitializeList()	אין	Void	מאתחלת רשימה התחלתית של מטרות כלליות

SaveItems()	אין	Void	שומרת את רשימת המטרות הכלליות בקובץ
ProTargetNum()	אין	double	מחזירה את הערך המספרי עבור מטרת החלבון היומית
CalTargetNum()	אין	Int	מחזירה את הערך המספרי עבור מטרת הקלוריות היומית
CalculateAndGetTargetString(int? TDEEOrWeight, double ate, TargetData targetData)	TDEEOrWeight: נדרש לחישוב המטרות ate: כמות שנאכלה targetData: נתונים עבור המטרה	string	מחשבת את המטרה היומית הרצויה ומחזירה מחרוזת להצגת המטרה

MyProfileService

מחלקה זו אחראית על ניהול הנתונים האישיים, היא מכילה בתוכה מבנים שיעזרו לניהול זה,

מחלקה זו יורשת מMyService ותרחיב את הפונקציונליות שלה

מבנים

Gender (enum)

מייצג מין: Male / Female

Activity (enum)

מייצג את רמת הפעילות:

Inactive = 1200,

PartlyActive = 1375,

Active = 1550,

VeryActive = 1725

הערך שניתן הינו לפי מקדם פעילות המשפיע על שריפת הקלוריות היומית.

MoreValues (struct)

מיועד להתנהלות עם נתונים נוספים שאינם במבנה של MyValue.

MyGender מסוג Gender - מין המשתמש

MyActivity מסוג Activity – רמת הפעילות של המשתמש

BloodPressure מסוג bool – נקבע לtrue במידה ולמשתמש יש מחלה של לחץ דם (לחץ דם גבוה, נמוך..)

Diabetes מסוג bool – נקבע לtrue במידה ולמשתמש יש מחלה של סכרת(סכרת סוג 1, סוג 2..)

תכונות

שם המאפיין	סוג	תיאור
MyValues	רשימת אובייקטים מסוג MyValue	רשימת נתונים אישיים
FileMoreValues	string	כתובת הקובץ המכיל את הנתונים הנוספים
Activitys	רשימת אובייקטים מסוג Activity	רשימה זו נועדה עבור מעבר קל על האובייקטים האפשריים מסוג Activity
MoreValuesItem	MoreValues	אובייקט המכיל את הערכים לשמירה
MyGender	Gender	מין המשתמש
MyActivity	Activity	רמת פעילות המשתמש
BMI	double	ערך ה BMI מדד של מסת הגוף(Body Mass Index)
TDEE	Int	*ערך הTDEE* *(Total Daily Energy Expenditure)* מדד להוצאת האנרגיה היומית הכוללת
MyBloodPressure	bool	פרמטר בוליאני עבור מחלת לחץ דם
MyDiabetes	bool	פרמטר בוליאני עבור מחלת סכרת
MALE_S, FEMALE_S	string	מחרוזות המייצגות מין
WEIGHT_S, AGE_S, HEIGHT_S	string	מחרוזות המייצגות נתונים אישיים מספריים

בנאים

שם הבנאי	פרמטרים	תיאור
MyProfileService()	אין	השמה של FileS ו FileMoreValues לכתובות הרצויות להמשך הפעילות במחלקה.

ממשקים ציבוריים(Public Interfaces)

שם השיטה	פרמטרים	מחזירה	תיאור
LoadItemsToTheList(string itemJson)	itemJson: מחרוזת JSON לטעינת נתונים אישיים	Void	אתחול נתונים אישיים כפי שנשמרו בקובץ

LoadItems()	אין	void	בנוסף למתואר במחלקת האב טוען נתונים נוספים ומחשב BMI ו TDEE
InitializeList()	אין	Void	מאתחלת רשימה התחלתית של נתונים אישיים
SaveItems()	אין	Void	שומר את הנתונים האישיים בקובץ ומחשב BMI ו TDEE
IsMyGenger(string gender)	gender: מחרוזת לבדיקה	bool	מחזירה true אם המחרוזת שהתקבלה היא מין המשתמש.
IsMyActivity(Activity activ)	activ: אובייקט מסוג Activity	bool	מחזירה true אם האובייקט שהתקבל שווה לרמת הפעילות של המשתמש.
UpdateGender(string? genderString)	genderString: מחרוזת	void	מעדכנת את מין המשתמש ומחשבת את ערך הTDEE
UpdateActivity(Activity activity)	activity: אובייקט מסוג Activity	Void	מעדכנת את רמת הפעילות של המשתמש ומחשבת את ערך הTDEE
Weight()	אין	int	מחזירה את משקל המשתמש
Age()	אין	int	מחזירה את גיל המשתמש
Height()	אין	Int	מחזירה את גובה המשתמש
MeaningOfBMI()	איו	string	מחזיר את משמעות ערך הBMI של המשתמש(משקל תקין/ עודף..)
IsNullOrZero(List<double ?> doubles)	רשימה של double או null	Boolean	מחזירה true אם אחד או יותר מהאובייקטים שווה לאפס או null
RecommendationsString (string recommendations)	Recommendations : מחרוזת עבור שם ההמלצה הרצויה	string	מחזירה את המלצה הרצויה בהתאם למחלות המסומנות

7. שכבת ההצגה(Presentation Layer)

שכבה זו מציגה ממשק משתמש (UI)שיאפשר לו ליצור אינטראקציה עם המערכת. כמו כן, שכבה זו

מתממשקת עם שכבת הלוגיקה, בכדי לקבל ממנה שירותים (שהיא עצמה מספקת ישירות או בעקיפין באמצעות שכבת בסיס-הנתונים. (בטפסים שאינם מהווים חלונות צצים (Pop Up) יעמוד לרשות המשתמש סרגל כלים עליון לניווט בין החלונות.

תוכנה זו נכתבה בעזרת blazor hybrid,

עקרונות מנחים בשכבת ההצגה:

- ממשק המשתמש יהיה נוח ופשוט לשימוש.
- שמירה על הפרדה בין לוגיקת הקוד לתצוגה.
- פירוק הממשק לרכיבים הניתנים לשימוש חוזר
- שימוש ב State Management- שימוש במנגנונים מתאימים עבור תקשורת בין רכיבים וכדומה.
- טעינת נתונים בצורה אסינכרונית וניווט קל בין הדפים על מנת לשמור על חוויית משתמש חלקה
- שימוש בדפוסי עיצוב(Patterns Design)רלוונטיים כדי שהעיצוב של שכבה זו יהיה גמיש לשינויים ויעמוד ביעדי הנדסת תוכנה נוספים.

כעט נעבור על המחלקות השונות בשכבה זו.

Home

מחלקה זו אחראית על הצגת מעקב אחר תזונה והמלצות למשתמש וטעינת הנתונים עבור כל המערכת.

המחלקה מורכבת ממרכיבי UI המאפשרים הצגת נתונים למשתמש. היא מתקשרת עם המחלקות FoodService, FoodTodayService, GoalService, MyProfileService לטעינת הנתונים

תיאור מבני

כותרת: " Health App "

כותרת נוספת "Welcome to your health app. "

נתוני BMI: מוצג ערך ה BMI אם קיים, ובנוסף הערה על משמעות הערך.

נתוני TDEE: מוצג ערך ה TDEE אם קיים (שיעור שריפת קלוריות יומית)

פסי התקדמות עבור מטרות יומית: פס התקדמות אל עבר המטרות יומיות של קלוריות, חלבון, ושתית מים.

המלצות תזונה ופעילות במידת הצורך

כפתור שמירה: כפתור לשמירת כל הנתונים של המערכת.

ממשק משתמש (UI)

שימוש ברכיבים:

- < ProgressBar >: להצגת פסי התקדמות אל עבר המטרות.
- <button>: עבור שמירה

טעינת הנתונים: אם אין נתונים מוצגת הודעת "Loading…".

תכונות ופרמטרים

שם המאפיין	סוג	תיאור
FService	FoodService	מופע השירות לניהול נתוני המזון
FTService	FoodTodayService	מופע השירות לניהול נתוני המאכלים שנאכלו
GService	GoalService	מופע השירות לניהול המטרות הכלליות
MPService	MyProfileService	מופע השירות לניהול הנתונים האישיים
load	bool	משתנה עזר להצגת הודעת "Loading…".

ממשקים ציבוריים(Public Interfaces)

שם השיטה	תיאור
OnInitialized	אחראית על אתחול הדף טעינת הנתונים ורישום לשירות עדכון ממשק המשתמש בעת שינוי הרכיבים הרלוונטים.
UpdateState()	עדכון ממשק המשתמש(UI)
CaloreTargetString	מחזירה מחרוזת המתארת את המטרה הקלורית
ProTargetString	מחזירה מחרוזת המתארת את מטרת החלבון
WaterTargetString	מחזירה מחרוזת המתארת את מטרת השתיה
Save	שומרת את הנתונים של כל המערכת
Dispose	מימוש הפונקציה להסרת המנוי (לעדכון ממשק המשתמש בעת הצורך)

שירותים בשימוש

FoodService:

השירות מספק שיטות לניהול מזון:

- LoadItems
- SaveItems

FoodTodayService:

השירות מספק שיטות לניהול המאכלים שנאכלו

- NumGlassBeforNumOfDays
- TotalValuesBeforNumOfDays
- LoadItems
- SaveItems

MyProfileService:

השירות מספק את הנתונים הדרושים לעמוד זה:

- BMI
- TDEE
- MyBloodPressure
- MyDiabetes

השירות מספק שיטות לקבלת ושמירת הנתונים:

- MeaningOfBMI
- RecommendationsString
- Weight
- LoadItems
- SaveItems

GoalService:

השירות מספק את הנתונים הדרושים לעמוד זה:

- CalTargetData

- ProTargetData

השירות מספק שיטות לקבלת ושמירת הנתונים:

- CalculateAndGetTargetString
- Save
- LoadItems

ProgressBar

מחלקה זו אחרית על תצוגת פס התקדמות עבור המטרות היומיות

המחלקה מורכבת מרכיבי UI לתצוגה ברורה למשתמש.

תיאור מבני

כותרת: שם המטרה

פס התקדמות

תיאור מילולי של פס ההתקדמות.

ממשק משתמש (UI)

שימוש ברכיבים:

- : עבור תצוגת הפס וההתקדמות עליו.

תכונות

שם המאפיין	סוג	תיאור
Value	double	הערך עבור ההתקדמות
Max	double	הערך המקסימלי המסמל 100% מהפס
Target	string	שם המטרה
TargetString	string	מחרוזת לתיאור ההתקדמות אל המטרה

ממשקים ציבוריים(Public Interfaces)

שם השיטה	תיאור
CalcolatePercent	מחשבת את אחוז ההתקדמות

Profile

מחלקה זו אחראית על הצגת עמוד פרופיל המשתמש. המשתמש מזין בו את המידע האישי.

המחלקה מורכבת ממרכיבי UI המאפשרים איסוף נתונים מהמשתמש. היא מתקשרת עם המחלקה MyProfileService לאחסון וטעינת הנתונים

תיאור מבני

כותרת: "Your Profile"

הוראה: "Please enter your personal information".

נתונים אישיים מספריים: מוצגים ערכים מרשימת MPService.MyValues. כל ערך מכיל שדה קלט עם הצגת השם והיחידות הרלוונטיות.

בחירת מין: אפשרויות בחירה: "Female" / "Male". מסומנת ברירת המחדל הקיימת בשירות (אם קיימת).

בחירת רמת פעילות: מוצגות אפשרויות עבור רמת הפעילות מרשימת MPService.Activitys. מסומנת ברירת המחדל הקיימת בשירות (אם קיימת).

צ'קבוקסים עבור מצבים בריאותיים: סכרת (Diabetes), לחץ דם גבוה.(Blood Pressure)

ממשק משתמש (UI)

שימוש ברכיבים:

- <input>: לקליטת ערכים, בחירת רדיו וצ'קבוקסים
- <label>: לתיאור הרכיבים

לולאת foreach להצגת הערכים באופן דינמי

טעינת הנתונים: אם אין נתונים מוצגת הודעת "Loading…".

תכונות

שם המאפיין	סוג	תיאור
MPService	MyProfileService	מופע השירות לניהול נתוני הפרופיל
Genders	מערך של string	מייצג את אפשרויות המגדר

ממשקים ציבוריים(Public Interfaces)

שם השיטה	תיאור
OnChangeGender	מעדכנת את המגדר הנבחר ע"י קריאה לשירות UpdateGender
isMyGenger	בודקת האם המגדר הנוכחי מתאים למגדר הנתון

isMyActivity	בודקת האם רמת הפעילות הנוכחית מתאימה לרמת הפעילות הנתונה
GetStringOfMV	מחזירה מחרוזת להצגת הגדרת הנתונים האישיים המספריים – שם ויחידות
OnChangeActivity	מעדכנת את רמת הפעילות הנבחרת ע"י קריאה לשירות UpdateActivity
Save	שומרת את הנתונים המעודכנים ע"י קריאה לשירות Save

שירותים בשימוש

MyProfileService:

השירות מספק את הנתונים הדרושים לעמוד זה:

- רשימת ערכים מספריים.(MyValues)
- אפשרויות הפעילות.(Activitys)
- המגדר הנוכחי והמצב הבריאותי (סכרת/לחץ דם).

השירות מספק שיטות לעדכון ושמירת הנתונים:

- UpdateGender(string gender)
- UpdateActivity(Activity activity)
- SaveItems()
- IsMyGenger(string gender)
- IsMyActivity(Activity activity)

Goals

מחלקה זו אחראית על הצגת עמוד המטרות האישיות. המשתמש מסמן בו את מטרותיו מתוך המטרות הקיימות.

המחלקה מורכבת ממרכיבי UI המאפשרים איסוף נתונים מהמשתמש. היא מתקשרת עם המחלקה GoalService לאחסון וטעינת הנתונים

תיאור מבני

צ'קבוקסים עבור המטרות. המשתמש יסמן את המטרות שלו.

כפתור שמירה: כפתור לשמירת הנתונים.

ממשק משתמש (UI)

שימוש ברכיבים:

- <input>: לצ'קבוקסים
- <label>: לתיאור הרכיבים
- <button>: עבור שמירת הנתונים

לולאת foreach להצגת הערכים באופן דינמי

טעינת הנתונים: אם אין נתונים מוצגת הודעת "Loading…".

תכונות

שם המאפיין	סוג	תיאור
GService	GoalService	מופע השירות לניהול נתוני המטרות

ממשקים ציבוריים(Public Interfaces)

שם השיטה	תיאור
Save	שמירת הנתונים המעודכנים ע"י קריאה לשירות Save

שירותים בשימוש

GoalService:

השירות מספק את הנתונים הדרושים לעמוד זה:

- רשימת המטרות הכלליות Goals.

השירות מספק שיטות לעדכון ושמירת הנתונים:

- Save ()

EatToday

מחלקה זו אחראית על הצגת עמוד ניהול מאכלים אישיים. המשתמש מזין בו את המאכלים שנאכלו ומעדכן כוסות מים ששתה.

המחלקה מורכבת ממרכיבי UI המאפשרים קליטת נתונים משמשתמש. היא מתקשרת עם המחלקות FoodTodayService, FoodService לאחסון עדכון וטעינת הנתונים.

תיאור מבני

כותרת: "What did you eat today "

טקסט: מידע על מספר הכוסות שנשתו היום,

כפתור הוספה: כפתור להוספת כוס שתיה

טקסט: רשימת המאכלים והכמויות שנאכלו

תיבת טקסט: תיבה לחיפוש ברשימת המאכלים

רשימה נפתחת: רשימה נפתחת המכילה את המאכלים המתאימים לסינון

תיבת טקסט: תיבה להכנסת הכמות שנאכלה

כפתור הוספה: כפתור להוספת המאכל והכמות הנבחרים

כפתור שמירה: כפתור לשמירת הנתונים.

ממשק משתמש (UI)

שימוש ברכיבים:

- <input>: לקליטת ערכים
- <select>, <option>: עבור רשימת המאכלים הנפתחת.
- <button>: עבור הוספה ושמירת הנתונים.

לולאת foreach להצגת הערכים באופן דינמי

תכונות

שם המאפיין	סוג	תיאור
FTService	FoodTodayService	מופע השירות לניהול נתוני המאכלים שנאכלו
FService	FoodService	מופע השירות לניהול נתוני המזון.
NewFoodItem	string	משתנה עזר לזיהוי פריט המזון הנבחר
Amount	string	הכמות שנאכלה

SearchTerm	string	מחרוזת עבור חיפוש מאכלים
FilteredFoodItems	IEnumerable<FoodItem>	רשימת המאכלים לאחר סינון ע"פ החיפוש

ממשקים ציבוריים(Public Interfaces)

שם השיטה	תיאור
DrenkWater	מעדכנת את כוסות המים שנשתו ע"י קריאה לשירות AddGlassOfWater
AddFoodItem	ניהול הוספת מאכל לרשימת המאכלים שנאכלו. משתמש בשירות AddFood
Reset	מאפס שדות נדרשים לבחירת מאכל נוסף
Save	שומרת את הנתונים המעודכנים ע"י קריאה לשירות Save

שירותים בשימוש

FoodTodayService:

השירות מספק שיטות להצגה עדכון ושמירת הנתונים:

- FoodsBeforeNumOfDays(int days)
- NumGlassBeforNumOfDays(int beforNumOfDays)
- AddGlassOfWater()
- Save()
- AddFood(int foodId, int amount)

FoodService:

השירות מספק את הנתונים הדרושים לעמוד זה:

- רשימת המאכלים הקיימים במערכת

השירות מספק שיטות להצגת הנתונים:

- FoodTodayString(FoodToday food)

FoodsHistory

מחלקה זו אחראית על הצגת היסטורית האכילה והשתיה של המשתמש. המשתמש בוחר יום מתוך רשימה, והמערכת מציגה לו את נתוני האכילה והשתיה שלו עבור אותו יום.

מחלקה זו מורכבת ממרכיבי UI המאפשרים קליטת נתונים. היא מתקשרת עם המחלקה FoodTodayService, FoodService לטעינת הנתונים בנוסף היא מתקשרת עם MyService לקבלת שיטות להצגת הנתונים.

תיאור מבני

כותרת: " Foods history "

רשימה נפתחת: רשימת ימים לבחירה.

רשימת מאכלים: רשימת המאכלים שנאכלו עבור היום הנבחר.

שתיה: נתוני שתיה עבור היום הנבחר.

נתוני אכילה: כותרת ופירוט ערכים תזונתיים שנאכלו בסך הכל עבור היום הנבחר.

ממשק משתמש (UI)

שימוש ברכיבים:

- <select>, <option>: עבור רשימת הימים לבחירה.
- <li> להצגת המאכלים

לולאת foreach להצגת הימים והערכים באופן דינמי

תכונות

שם המאפיין	סוג	תיאור
FService	FoodService	מופע השירות לניהול נתוני הפרופיל
FTService	FoodTodayService	מייצג את אפשרויות המגדר
BeforeNumDay	int	מיצג את היום הנבחר ע"י המשתמש
FoodsForDay	List<FoodToday>	רשימת המאכלים עבור היום הנבחר
Pro, Fat, Carbo, Energy, Sodium, Sugars	string	מייצגים את מחרוזת הערכים התזונתיים עבור היום הנבחר

ממשקים ציבוריים(Public Interfaces)

שם השיטה	תיאור

OnInitializedAsync	אחראית על אתחול הדף טעינת הנתונים ורישום לשירות עדכון ממשק המשתמש בעת שינוי הרכיבים הרלוונטים.
UpdateState()	עדכון ממשק המשתמש(UI)
Dispose	מימוש הפונקציה להסרת המנוי (עדכון ממשק המשתמש בעת הצורך)
OnSelectDay	מעדכנת נתונים בהתאם, לאחר בחירת יום
FoodsValueString	מבצעת השמה עבור מחרוזות הערכים התזונתיים
Water	מחזירה מחרוזת עבור נתוני השתיה ביום הנבחר
StringForDay	מחזירה מחרוזת להצגת היום לבחירה
FoodTodayString	מחזירה מחרוזת להצגת מאכל
ShowDetails	מקפיצה חלון להצגת הנתונים עבור מאכל

שירותים בשימוש

FoodService:

השירות מספק שיטות להצגת הנתונים:

- SumFoodsValues(List<FoodToday> FoodsForDay)
- FoodTodayString(FoodToday food)
- FoodDetailsString(FoodToday food)
- FoodTitleForAmount(FoodToday food)

FoodTodayService:

השירות מספק שיטות לקבלת הנתונים הדרושים:

- FoodsBeforeNumOfDays(int days)
- NumGlassBeforNumOfDays(int BeforeNumDay)

MyService:

השירות מספק שיטות להמרה ולהצגת הנתונים:

- StringToInt(object obj)
- שיטות להצגת הערכים התזונתיים: לדוג' EnergyString(int Energy)

FoodsList

מחלקה זו אחראית על הצגת המאכלים הקיימים במערכת

המחלקה מורכבת ממרכיבי UI המאפשרים הצגת נתונים בצורה חלקה ברורה למשתמש. היא מתקשרת עם המחלקה FoodService לאחסון וטעינת הנתונים ועם המחלקה AddFood עבור הוספת נתונים.

תיאור מבני

כותרת: "Foods list "

הוראה: "click for show values per 100 gram ".

כפתור הוספה: כפתור לפתיחת טופס להוספת מאכל אישי.

תיבת חיפוש: תיבת טקסט להזנת מחרוזת לחיפוש ברשימת המאכלים

רשימת מאכלים: רשימה של המאכלים הקיימים במערכת.

חלון ערכים תזונתיים: בעת לחיצה על פריט מהרשימה יפתח חלון המציג ערכים תזונתיים עבור המאכל הנבחר.

ממשק משתמש (UI)

שימוש ברכיבים:

- <input>: לקליטת ערכים
- <button>: עבור פתיחת טופס להוספת מאכל חדש
- <AddFood> טופס להוספת מאכל

לולאת foreach להצגת הערכים באופן דינמי

תכונות

שם המאפיין	סוג	תיאור
FService	FoodService	מופע השירות לניהול נתוני המאכלים
IsModalOpen	bool	משתנה האחראי לפתיחת טופס להוספת מאכל
SearchFood	string	מחרוזת לחיפוש ברשימת המאכלים
FilteredFoodItems	IEnumerable<FoodItem>	רשימת המאכלים לאחר סינון

ממשקים ציבוריים(Public Interfaces)

שם השיטה	תיאור

ShowDetails	מקפיצה חלון להצגת הנתונים עבור מאכל
AddEatItem	פותחת טופס להוספת מאכל חדש
HandleModalStateChanged	אחראית לסנכרון המשתנה המורה על פתיחת הטופס

שירותים בשימוש

FoodService:

השירות מספק את הנתונים הדרושים לעמוד זה:

- Foods - רשימת המאכלים הקיימים במערכת

השירות מספק שיטות להצגה, עדכון ושמירת הנתונים:

- FoodTitleForAmount(FoodToday food)
- FoodDetailsString(FoodToday foodT)

AddFood

מחלקה זו אחראית על הצגת טופס להוספת פריט מזון אישי חדש למערכת. הטופס מאפשר למשתמש להזין ערכים תזונתיים עבור מזון חדש (ערכים ל 100 גרם).

המחלקה תומכת באימות נתונים, הצגת שגיאות והוספת הפריט לרשימה האישית של המשתמש באמצעות שירות FoodService.

תיאור מבני

המחלקה מציגה טופס בתוך חלון קופץ(Modal) .

- כותרת: "add new food. values per 100 g" .
- שדות לקליטת ערכים משמשתמש
  - שם המזון
  - אנרגיה (קלוריות)
  - חלבון (גרם)
  - שומן כולל (גרם)
  - פחמימות (גרם)

  - סוכרים (גרם)
  - נתרן (מ"ג)
- כפתור לאישור ההוספה
- כפתור לסגירת הטופס

ממשק משתמש (UI)

שימוש ברכיבים:

- <InputText >: לקליטת שם המאכל מהמשתמש.
- <InputNumber >: לקליטת ערכים מהמשתמש
- <button>: עבור הוספת המאכל וסגירת החלון
- <label> להצגת שם הערך

הטופס מוצג בתוך חלון קופץ (Modal) כאשר הפרמטר IsOpen מוגדר ל-true.

לאחר קלט תקין של המשתמש, הפריט נוסף לרשימת הפריטים האישיים ונשמר בקובץ

תכונות ופרמטרים

שם המאפיין	סוג	תיאור
FService	FoodService	מופע השירות לניהול נתוני המזון
Food	FoodItem	אובייקט לייצוג פריט המזון החדש
IsOpen	bool	מציין האם החלון פתוח
IsOpenChanged	EventCallback	משדר שינוי במצב הפתיחה של החלון

ממשקים ציבוריים(Public Interfaces)

שם השיטה	תיאור
Close	סוגרת את החלון ע"י קריאה IsOpenChanged.InvokeAsync(false)
HandleValidSubmit	מטפלת בהגשה תקינה

שירותים בשימוש

:FoodService

השירות מספק שיטות לניהול פרטי המזון

- :AddPersonalFood(FoodItem food)
- :SaveItems()

- GetFreeId()

אימות נתונים

המחלקה עושה שימוש ברכיב EditForm בשילוב עם:

- DataAnnotationsValidator: אימות המבוסס על ה-Data Annotations של המחלקה FoodItem.
- ValidationSummary: הצגת סיכום השגיאות באופן מרוכז.

8. דפוסי עיצוב

Singleton

תפקיד תבנית זו הוא לוודא שקיים רק מופע אחד של המחלקה. כך נמנעות טעויות לוגיות שיכולות להיווצר כתוצאה מיצירה של כמה מופעים עבור מחלקות אלו.

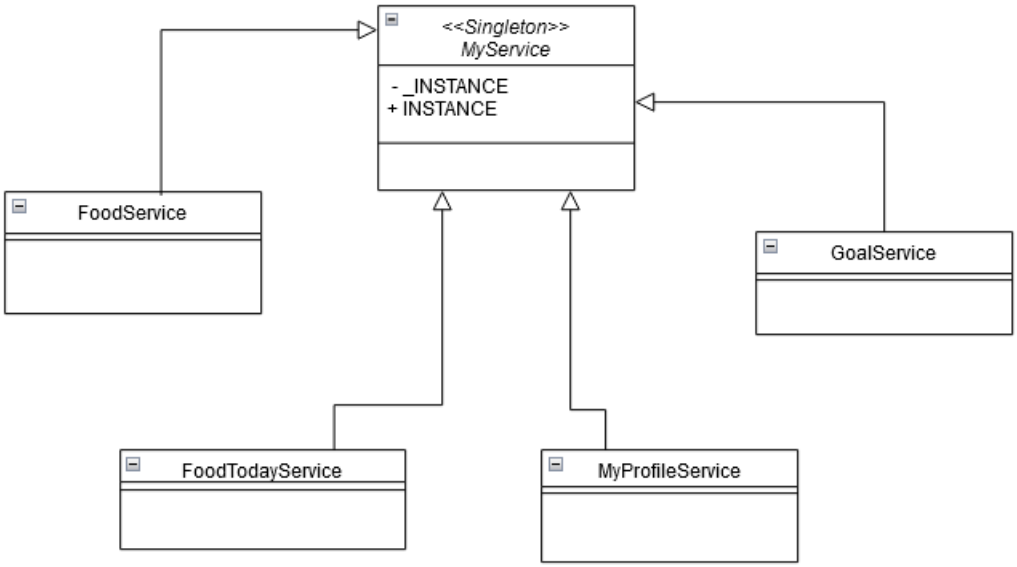
במערכת זו נעשה שימוש בדפוס עיצוב Singleton במחלקת אב גנרית (<MyService<T) עבור שימוש שירותים. מחלקה זו משמשת כבסיס לכל השירותים והוא משתמשת במנגנון Lazy Initialization על מנת ליצור את המופע היחיד בצורה עצלה.

יתרונות בשימוש בצורה זו:

- גמישות הקוד, מאפשרת יצירת שירותים חדשים בקלות.
- חיסכון במשאבים, מופע יחיד לכל מחלקה.

בכל מחלקה, על מנת להשתמש במופע של המחלקה ClassNameService היורשת מMyservice נדרש ליצור מופע בצורה הבאה:

```
ClassNameService CNService = ClassNameService.Instance;
```



MVVM (Model-View-ViewModel)

דפוס עיצוב הנועד לביצוע הפרדה בין הלוגיקה העסקית לבין חוויית המשתמש. הפרדה זו הינה בת 3 שלבים:

Model: מכיל את הנתונים והלוגיקה העסקית של המערכת

View: מציג את המידע למשתמש ומקבל את הקלט שלו.

ViewModel: מספק את הלוגיקה שמחברת בין ה View ל Model.

שימוש בדפוס עיצוב זה בעל יתרונות רבים כגון תחזוקה, יכולת הרחבה עתידית של המערכת, גמישות בפיתוח ועיצוב.

במערכת זו נעשה שימוש בדפוס זה באופן הבא:

מערכת זו פותחה בBlazor Hybrid

המערכת מחולקת לדפי razor ולדפי cs.

דפי razor מכילים 2 חלקים. בחלק הראשון מופיע קוד בHTML. חלק זה הינו ה-View. בחלק השני ישנו קוד בשפת #c המכיל לוגיקה עבור הצגת הנתונים. חלק זה שייך ל- ViewModel.

דפי ה cs מכילים מחלקות השייכות ל Model ומחלקות המכילות Model ו ViewModel

Observer

דפוס עיצוב זה מאפשר לרכיבים לתקשר ביניהם בצורה רופפת, כלומר אובייקט אחד יכול להודיע על שינויים שהיו בו לשאר האובייקטים הרשומים אליו.

במערכת זו השירות מתנהל ע"י מחלקת האב MyService המאזינה לשינויים, מחלקות השירותים היורשות ממנה, במידת הצורך, קוראות לפונקציה האחראית לפרסום אירועים. האירועים מאפשרים לרכיבי Blazor המנויים לשירות להאזין לשינויים במצב ולבצע עדכון של ה UI על ידי קריאה ל StateHasChanged.

בעת השמדת הרכיב, המנוי לאירוע מוסר כדי למנוע דליפות זיכרון בעזרת שיטת **Dispose.**

יתרונות הObserver- כפי שמנוהלות במערכת זו

- מניעת כפילויות, שכן רכיב מבצע את ניהול המצב וכל הרכיבים הנדרשים מאזינים ומגיבים לעדכונים. כך הקוד נשאר נקי ומסודר.
- פשטות וגמישות, כל רכיב יכול להתעדכן על פי הצורך בעת שינוי המצב לא צורך בהעברה ישירה של נתונים בין רכיבים

9. שינויים עתידיים אפשריים

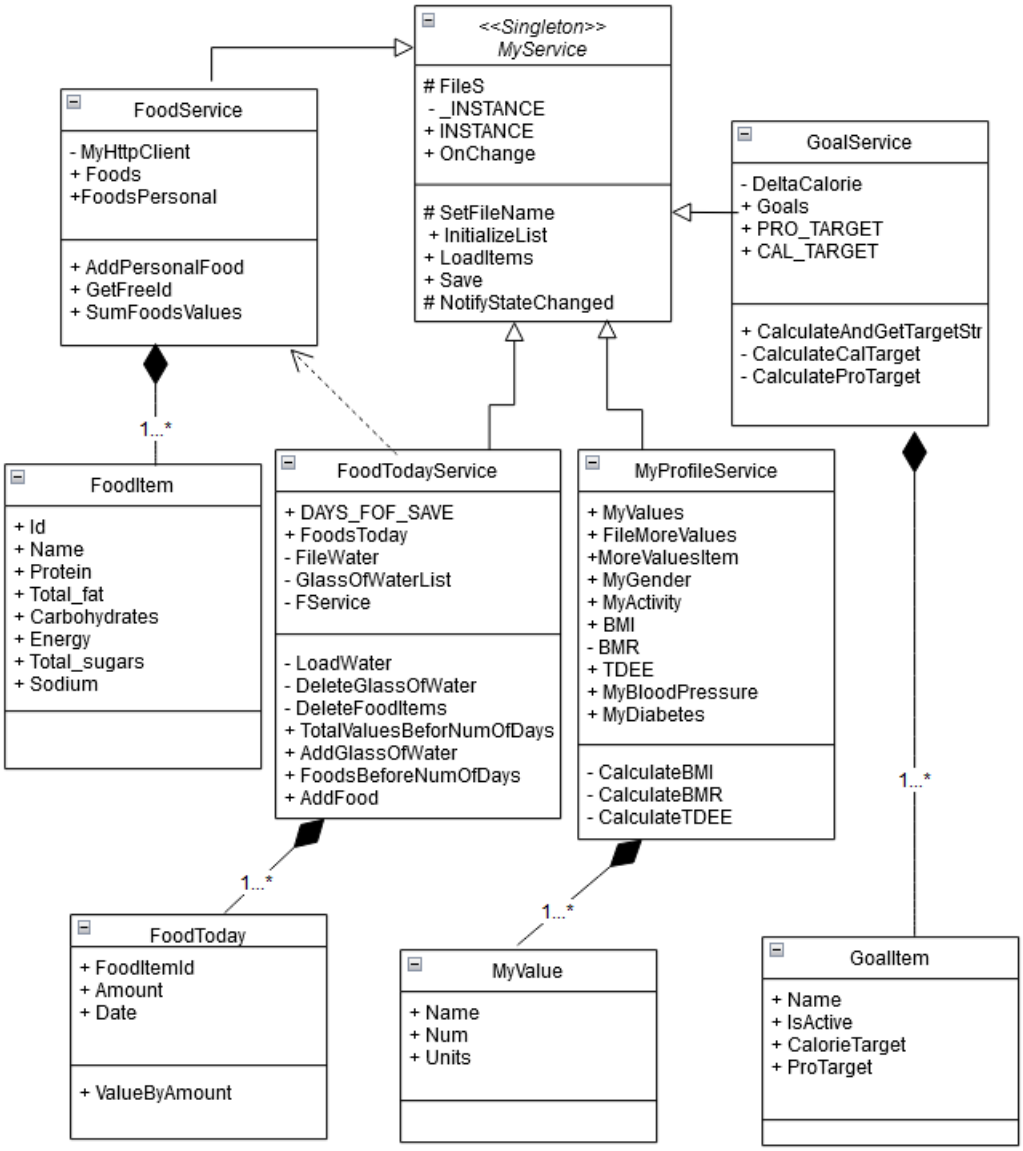
מערכת זו תוכננה ועוצבה כך ששינויים עתידיים יהיו קלים ליישום. נביא כמה דוגמאות כיצד זה בא לידי ביטוי ואליו שינויים ניתן לבצע.

- כל שכבה אחראית על תחום ספציפי, במידה ונרצה להחליף את שיטת המימוש של אחת השכבות נוכל לעשות זאת ללא החלפה של המימוש בשתי השכבות האחרות. לדוגמא אם נרצה להחליף את מסד נתונים מקבצי Json לפורמט אחר, נוכל לעשות זאת מבלי לפגוע בשכבות האחרות
- לאורך התוכנית הוגדרו קבועים עבור ערכים מסוימים (וגם מחרוזות). קבועים אלו יאפשרו לנו לשנות את הערכים במקום אחד בלבד. לדוגמא אם בעתיד נרצה לשנות את מספר הימים לשמירת ההיסטוריה למעל 30 ימים נוכל לעדכן את ערך DaysForSave.
- מימוש שיטת observer במחלקה MyService. בדרך זו, במידה וידרש להרחיב את המערכת (לדוגמא עבור מחלקות שירותים נוספים)דפוס עיצוב זה כבר קיים במחלקת האב, ולכן הרחבה עתידית תהיה פשוטה יותר .
- קובץ json הנמצא בשרת- בדרך זו עדכון המאכלים מתבצעים ע"י שינוי הקובץ. ישנה גם אופציה להוספת מאכלים חדשים. עדכון זה יתבצע רק בקובץ. בצורה כזו נוכל לעדכן את רשימת המאכלים עבור כל משתמשי הקצה בדרך קלה ופשוטה.

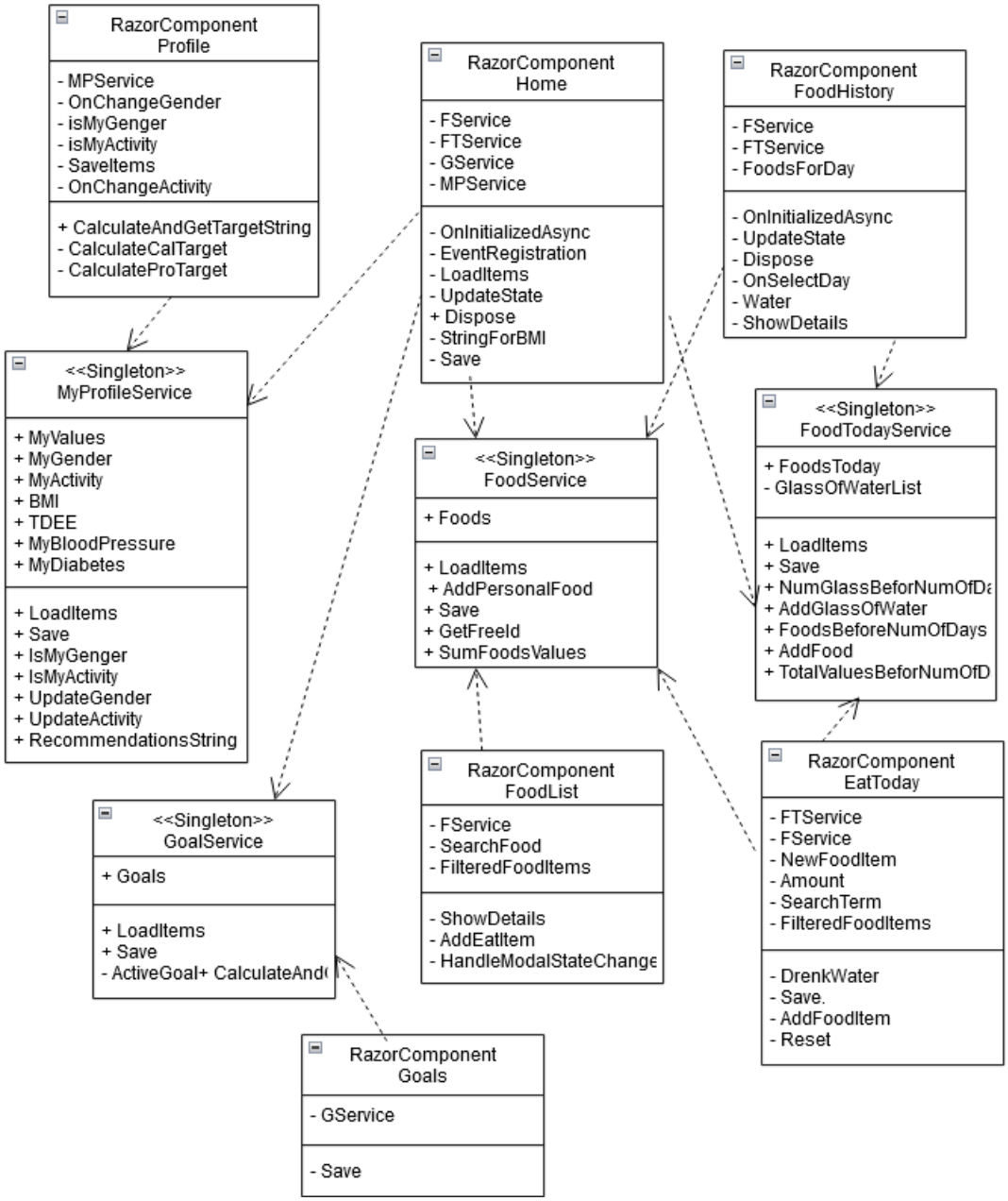
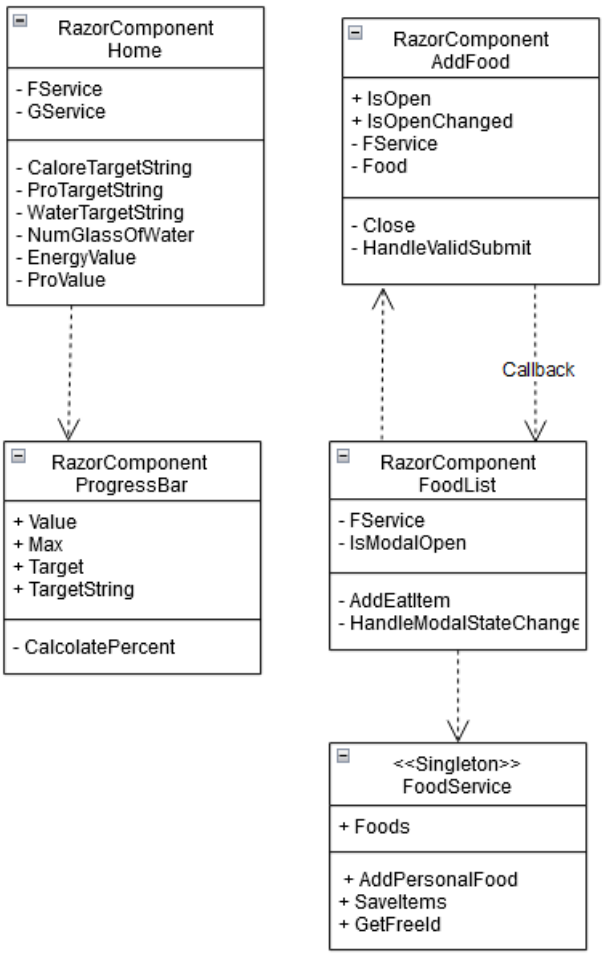
10. דיאגרמות

class diagram

דיאגרמת מחלקות Domain Classes



דיאגרמת מחלוקות ממשק המשתמש ושירותים



דיאגרמת מחלקות לממשקי משתמש נוספים (UI Classes):